

DESIGN OF HIGH SPEED APPROXIMATE MAC FOR MACHINE LEARNING APPLICATIONS

Abstract

Handling large and complex calculations in machine learning can be tough, especially when high performance and low power are both necessary. Traditional digital multipliers and accumulators carry out exact arithmetic operations, but these often create delays, take up more space, and consume more power. This creates serious limitations in real-time, resource-limited settings like edge devices and embedded AI systems.

This project shows the design and implementation of a high-speed Approximate Multiply-Accumulate (MAC) unit using Verilog HDL. The suggested architecture uses an Approximate Multiplier combined with an error-tolerant adder structure to achieve faster calculations with less logic complexity. By accepting a small loss in accuracy, the design significantly boosts speed and power efficiency.

We simulated and verified the overall MAC design using ModelSim and prepared it for ASIC implementation with Cadence. It was also integrated with MATLAB to use the Verilog design for image processing tasks. The results indicate that the proposed approximate MAC delivers higher throughput and lower delays than traditional exact MAC units. Therefore, this design fits well in machine learning and signal processing applications where speed and energy efficiency are prioritized, even if it requires a trade-off with accuracy.

Acknowledgements

We wish to express our sincere thanks and deep sense of gratitude to our project guide, Dr. Sasipriya P, Professor, School of Electronics Engineering, for her consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to Dean Dr. Ravi Sankar A, Associate Dean (Academics) Dr. Mohana Prasad K, & Associate Dean (Research) Dr. Chitra K, of the School of Electronics Engineering, VIT Chennai, for extending the facilities of the School towards our project and for his unstinting support.

We express our thanks to our Head of the Department Dr. Jagannath M for his support throughout the course of this project.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

Chapter 1

Introduction

The fast expansion of artificial intelligence and machine learning has created a strong need for hardware that performs well and uses little energy. These technologies rely heavily on repeated math tasks like multiplication and accumulation, which support things like neural networks and signal processing in images. Designers face tough issues in making systems that run quickly yet keep power low and costs down, particularly for small devices with tight limits on what they can handle.

Standard units for multiplication and accumulation do precise calculations. They deliver accuracy but end up slower with more space and energy used. Still, many tasks in machine learning and signal processing can handle small mistakes along the way. Such errors hardly change the end results. This allows for approaches in computing that add those minor flaws on purpose. In turn, they bring big improvements in how fast things go, how little power they take, and how small the hardware stays.

This effort involved creating and building a quick approximate unit for multiplication and accumulation in Verilog code. The setup uses a special Wallace multiplier that allows some approximation, paired with an adder that tolerates errors. Together, they boost performance while keeping accuracy at a usable level. Tests ran through simulation in ModelSim, then synthesis for chip making with Cadence tools. MATLAB helped check it out for handling images, showing how it works in real time for those uses.

Overall, the goal here shows how methods with approximate math can lift speed and cut power needs a lot. They do this without hurting how reliable the whole system is. That makes them a good fit for machine learning setups, signal processing, and embedded devices that run on low power.

Chapter 2

Literature Survey

2.1 Designing a Performance-Centric MAC Unit with Pipelined Architecture for DNN Accelerators (Raut et al.)

This work introduces a multiply-and-accumulate unit designed for deep neural network accelerators. It relies on a pipelined architecture that draws from the coordinate rotation digital computer method. Such a setup helps counter the limited throughput often seen in recursive CORDIC configurations. Evidence points to a careful handling of the balance between power and area costs on one side and overall performance on the other. To pinpoint the right number of pipeline stages, a Pareto analysis comes into play. That analysis reveals five stages as offering stronger performance measures over the recursive option. The final design shows a smaller area-delay product along with greater throughput when set against recursive CORDIC multiply-and-accumulate units. On top of that, it delivers close to double the speed of Xilinx multiply-and-accumulate operations run on a Virtex-7 field-programmable gate array.

2.2 Quantization aware approximate multiplier and hardware accelerator for edge computing of deep learning applications (Reddy et al.)

This work proposes a quantization-aware approximate radix-4 Booth multiplier and a corresponding hardware accelerator designed specifically for power-restricted edge computing devices running deep learning applications. The design improves power and resource efficiency by moving the output truncation step from the final accumulation stage to the multiplier stage, which allows for smaller adders. The accelerator uses parallel processing elements and includes data-fetching optimizations like weight re-arrangement and ping-pong buffers to enhance data communication bandwidth and achieve high throughput on FPGAs for matrix multiplication workloads.

2.3 8-Bit Approximate Multiplier using Approximate Full Adder (Karthikeya et al.)

This study introduces three new approximate full adders, labeled DAFA1, DAFA2, and DAFA3. These designs fit into Wallace multiplier setups. The goal focuses on cutting power use in areas tolerant of errors, such as image processing or machine learning tasks. Each design adds approximation only at the rare input case of 111. At the same time, it keeps the carry output precise. This approach limits how errors spread and helps preserve overall accuracy levels. Among them, DAFA3 stands out for efficiency. It shows a clear drop in the power-delay product, or PDP. Plus, its mean error distance, known as MED, comes in much lower than some earlier low-power approximate adders. In this way, the design strikes a stronger balance. Hardware savings pair well with needed precision in computations.

2.4 Low-Power Digital Signal Processing Using Approximate Adders (Gupta et al.)

This paper introduces a design method for low power in digital signal processing. It relies on error resiliency to simplify logic at the transistor level. The focus is on the mirror adder cell. Researchers present five simplified versions of this cell. These versions cut down transistors and internal capacitances. Such changes allow for better voltage scaling. They also lead to major power reductions when compared to standard accurate adders. To show how well this works, the approximate adders get applied to the least significant bits in certain circuits. These include ones for image compression through DCT and IDCT, as well as video compression in MPEG encoders. The approximation methods here deliver better output quality than basic bit truncation techniques.

2.5 Input-Conscious Approximate Multiply-Accumulate Unit for Energy-Efficient Computing

This paper examines approaches to designing low-power systems in digital signal processing. It employs error resiliency techniques to reduce logic complexity at the transistor level. Particular attention centers on the mirror adder cell. Five simplified variants of this mirror adder are presented. Each variant reduces the number of transistors and internal capacitances. Such reductions facilitate lower voltage scaling. In turn, this results in substantial power savings compared to conventional accurate adders. To demonstrate effectiveness, these approximate adders are applied to the least significant bits within image compression circuits, such as those involving DCT and IDCT. Similar applications appear in video compression for MPEG encoders. Evidence indicates that these approximation methods yield superior output quality. They outperform simple bit truncation strategies.

2.6 qLUT: Input-Aware Quantized Table Lookup for Energy-Efficient Approximate Accelerators

This paper proposes replacing complex arithmetic with quantized lookup tables (LUTs) that map inputs to pre-computed outputs, achieving significant energy savings with minimal accuracy loss for error-tolerant applications. However, a key limitation of this approach is its scalability; maintaining larger LUTs for higher accuracy increases memory usage and lookup latency, creating a trade-off between precision and practical hardware overhead. This method offers a software-hardware co-design alternative to circuit-level approximation techniques but is constrained by the exponential growth in LUT size for high-input dimensionality.

2.7 Approximate Compressors for Error-Resilient Multiplier Design by Yang et al.

This paper presents novel approximate compressors tailored for use in the partial product reduction tree of multipliers. The main goal from the authors involves cutting down the power and area expenses tied to exact multipliers. They do this by swapping in approximate versions where it makes sense. These compressors aim to keep the mean error distance low. That helps maintain decent output quality in areas like image filtering. Perfect accuracy is not always needed there. Experiments show big gains in power and area savings. The effect on Peak Signal-to-Noise Ratio for images stays very small. Still, this method comes with a trade-off. Arithmetic precision drops a bit. That could limit things for tasks needing high accuracy. Scientific computing and financial work fall into that category. Overall, the approach sets up a useful way to design multipliers that save energy. It fits well for error-tolerant systems in embedded devices and multimedia tasks.

2.8 Approximate Radix-8 Booth Multipliers for Low-Power and High-Performance Operation

The paper presents a modified Booth encoding scheme that simplifies partial product generation to reduce hardware complexity and switching activity. By introducing controlled approximations in the encoding and reduction stages, the design achieves lower power consumption and shorter critical path delay compared to conventional Booth multipliers. The architecture is optimized for energy-efficient and high-speed computation, making it suitable for low-power embedded and DSP systems. However, it incurs minor accuracy loss, requiring careful error tuning and making it less ideal for precision-critical applications.

Chapter 3

Design of Approximate Multiplier

3.1 Overview

Research centers on creating a fast and power-saving approximate multiply-accumulate unit. This unit uses Verilog HDL. It targets machine learning and signal processing mainly. The approach combines a mixed design for approximate math with operand approximation based on inputs. Such a setup cuts power use and delay a lot. It does this while keeping accuracy loss small. The design steps come in order. First comes idea review and past studies. Then architecture setup follows. Next involves coding subparts in Verilog. Simulation and checks happen with ModelSim after that. ASIC synthesis uses Cadence Genus. Finally validation in MATLAB tests image processing in real scenarios. This whole process balances accuracy in calculations with hardware savings. It fits well with approximate computing ideas. Studies show this kind of balance could improve efficiency in demanding apps. Evidence points to real gains without big errors.

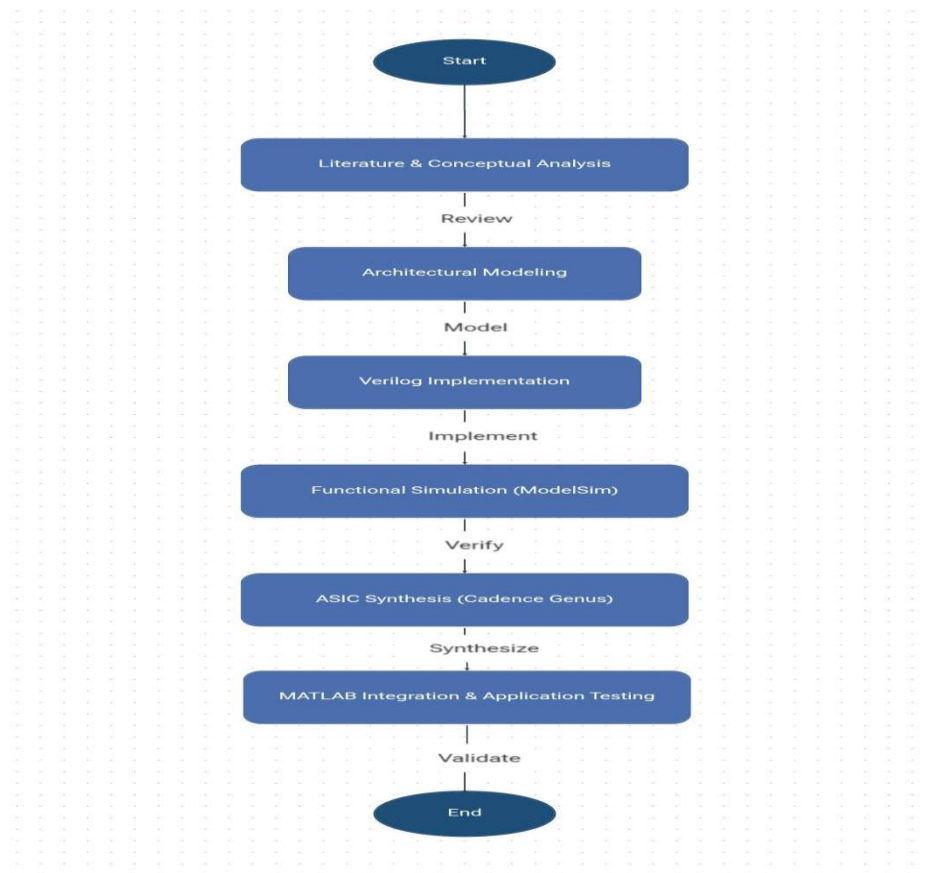


Figure 1 - Methodology Workflow

3.2 Proposed System Architecture

The proposed system, called ApproxMAC_AWM3, relies on a hierarchical structure built in Verilog. It incorporates various parameterized modules. Key components include the Register module, the ConditionalBlock, the approximate multiplier known as AWM3, and the ResultSelector. The inputs involve two 8-bit operands, X and Y. The output takes the form of a 20-bit value named Result. Future versions might incorporate MAC accumulation features.

This architecture for ApproxMAC_AWM3 forms a closely integrated sequence of modules. Together, they create an input-sensitive hybrid approximate MAC unit. To grasp how it works, one must trace the data and control signals through each phase, along with the choices made by the conditional elements. At the pipeline's start, the setup captures and holds the external operands. It does this by loading the 8-bit multiplicand X and multiplier Y into specialized Register modules at the clock's rising edge. These registers help synchronize timing throughout the datapath. They also shield the combinational logic from input-output fluctuations on the chip. Such isolation aids in achieving stable timing and cuts down on power use in sequential designs.

After registration, the operands move forward to two separate routes. One is the conditional path, sensitive to inputs. The other handles approximate multiplication. The ConditionalBlock carries out an input-driven approach, as outlined in the Input-Conscious AxMAC paper. It begins by detecting simple cases, like when any operand is zero. In those instances, it outputs zero right away and skips the multiplier. If one operand is precisely a power of two, the block calculates the product through a left shift of the other operand. The shift amount equals the log base two of that power-of-two value. This method swaps out a costly multiplication for a quick barrel shift operation.

When an operand falls short of being a pure power of two but stays within a set range suitable for approximation, the ConditionalBlock assigns it to the closest power-of-two equivalent. It uses the `approx_to_pow2` function for this mapping. Then, it shifts the accompanying operand accordingly. For all these conditional scenarios, the block sets the MA control signal high. It also generates a 16-bit conditional result, CR. This setup lets the design skip complete multiplication for many inputs. Evidence from the Input-Conscious study indicates coverage of about 43.8 percent of 8-bit operand combinations. As a result, switching drops sharply, and dynamic energy falls for typical input patterns.

Should none of these shortcuts fit, the MA flag stays low. The path then directs the registered operands to the approximate multiplier, either AWM3 or an updated Approx5 version for 8-bit operations. This multiplier follows an 8 by 8 Wallace tree to compress partial products. It starts with bitwise AND operations to form eight rows of partials. These undergo reduction via staged adders. Importantly, the less significant bits in each stage apply the designed approximate full adder, DAFA3. The higher significance bits, however, use precise full adders. This targeted approximation limits errors to low-value positions. It keeps carry propagation accurate in upper bits. Thus, overall error stays controlled.

The DAFA3 adder itself streamlines the sum computation. It drops XOR gates and adds a deliberate error only for the uncommon 111 input. Yet, it keeps the carry output exact. This approach shortens logic levels and cuts capacitance in the tree structure. Delay and power both decrease as a result. Error spread remains limited since

carries hold true. The reduction happens in stage_adder_dafa3 modules. These allow tuning of the approximate low-order bit count, APP_BITS. Designers can adjust the approximation level to balance accuracy against efficiency.

Once reduction completes, the multiplier yields a 16-bit product, MUL. This joins the conditional result CR and the MA flag to enter the ResultSelector. That unit acts as a straightforward multiplexer, driven by MA. It picks CR if MA is active, or MUL if not. The chosen 16-bit value then extends with zeros to reach 20 bits for the Result. This matches needs in downstream accumulators, such as those in MAC arrays or convolution flows.

The whole design supports easy modular testing. Individual parts, from registers and the conditional block to multiplier stages, adders, and the selector, can undergo unit tests in ModelSim. Custom testbenches target edge conditions, like zero values, pure powers of two, ranges open to approximation, and all-purpose operands. They also track switching behavior. The core idea behind the architecture involves substituting shifts for multiplication in a wide input range. It also limits approximation to minor bit positions. This aligns closely with aims in the Input-Conscious paper. Partial product creation and activity decrease. Fewer adders activate. Dynamic power shrinks. The critical path shortens in everyday scenarios. Accuracy holds up well enough for tasks in image processing and machine learning, where minor numerical shifts prove acceptable.

In operation, operands first load into register banks at the top level. This latching ensures proper sync with the clock. Next, the stored inputs feed into the ConditionalBlock. There, it decides between exact multiplication or approximation using bit shifts. The AWM3 then manages the rough multiplication. It builds on an approximate Wallace multiplier base. For lower significance zones, DAFA3 full adders come into play. Upper bits get standard exact adders. Finally, the ResultSelector merges the results. It switches between the conditional shift output and the AWM3 product. In this manner, the system gains flexibility in operation while upholding needed precision.

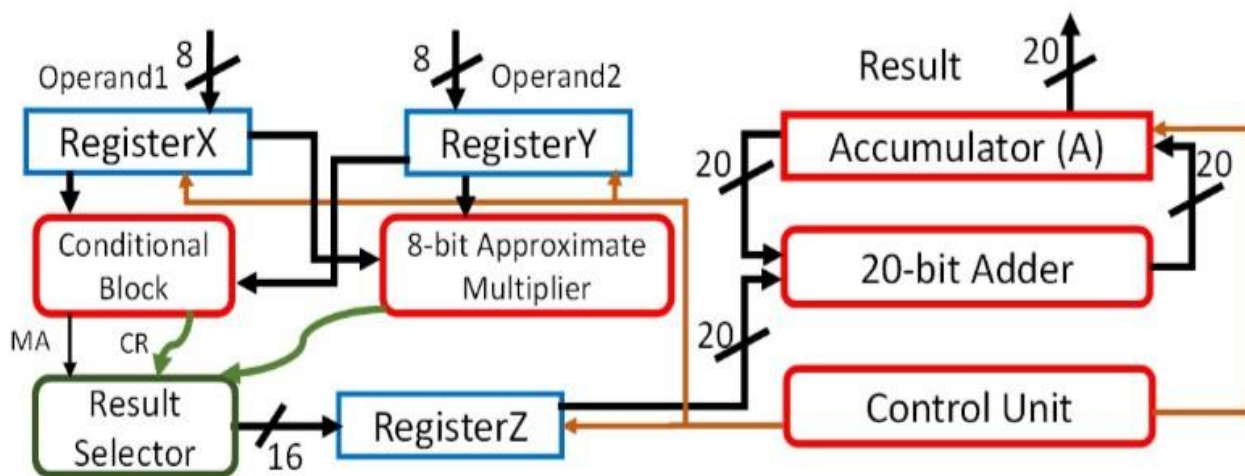


Figure 2 - Proposed Architecture

3.2.1 Register X and Register Y

The design incorporates two 8-bit registers at the outset. Register X and Register Y hold the input operands. These are Operand1 and Operand2. Evidence from the structure shows that these registers function as input

latches. They synchronize external data with the system clock. This setup ensures both operands reach the computation stage at the same time. They also stay stable throughout processing. Such isolation from combinational logic transitions appears to prevent glitches effectively. It helps maintain timing stability as well. Dynamic power consumption sees a reduction too. Outputs from these registers connect to the Conditional Block. They also link to the 8-bit Approximate Multiplier. This allows decisions for parallel processing.

3.2.2 Conditional Block

The **ConditionalBlock** plays a crucial role in achieving input-aware optimization. This block analyzes the nature of operands (X and Y) and dynamically decides whether to execute the multiplication or to perform a simplified shift operation based on certain conditions. The logic operates under three major cases:

1. **Zero Operand Detection:** If either input is zero, the block outputs zero directly, bypassing the multiplier to save power.
2. **Power-of-Two Operand Detection:** If an operand is an exact power of two, the multiplication is replaced by a logical left shift ($Y \ll \log_2(X)$), which produces an equivalent result with minimal delay and zero arithmetic cost.
3. **Approximation to Nearest Power-of-Two:** For non-power-of-two values, the input is mapped to its nearest power-of-two range using the `approx_to_pow2()` function. This controlled approximation introduces minor computational error but yields substantial delay and power benefits.

These input-aware mechanisms effectively reduce unnecessary switching activity in the multiplier, significantly cutting dynamic power consumption. The conditional block thus implements an intelligent control signal (**MA**) that dictates whether the AWM3 block should be activated or bypassed. This block is inspired by the **Input-Conscious AxMAC** framework and adapted for ASIC synthesis using Verilog HDL.

The Conditional Block in the proposed ApproxMAC architecture uses predefined operand ranges to determine when input values can be approximated to their nearest power-of-two equivalents, thereby replacing complex multiplication with simple shift operations. As shown in the operand approximation table, specific input intervals such as **6–11**, **28–39**, and **60–71** are mapped to **8**, **32**, and **64**, respectively. When an operand falls within one of these ranges, the block activates the approximation mode, substitutes the operand with its corresponding power-of-two value, and performs a left-shift operation instead of full multiplication. This strategy significantly reduces switching activity, power consumption, and critical path delay, while maintaining acceptable accuracy for error-tolerant applications such as machine learning and image processing.

Approximable Operand				Approximated Operand	
Binary		Decimal			
Min	Max	Min	Max	Decimal	Binary
0000 0000	0000 0000	0	0	0	0000 0000
0000 0001	0000 0001	1	1	1	0000 0001
0000 0010	0000 0011	2	3	2	0000 0010
0000 0100	0000 0101	4	5	4	0000 0100
0000 011X	0000 10XX	6	11	8	0000 1000
0000 11XX	0001 00XX	12	19	16	0001 0000
0001 11XX	0010 0XXX	28	39	32	0010 0000
0011 11XX	0100 0XXX	60	71	64	0100 0000
0111 11XX	1000 0XXX	124	135	128	1000 0000

Table 1 - Range specification for 8 bit

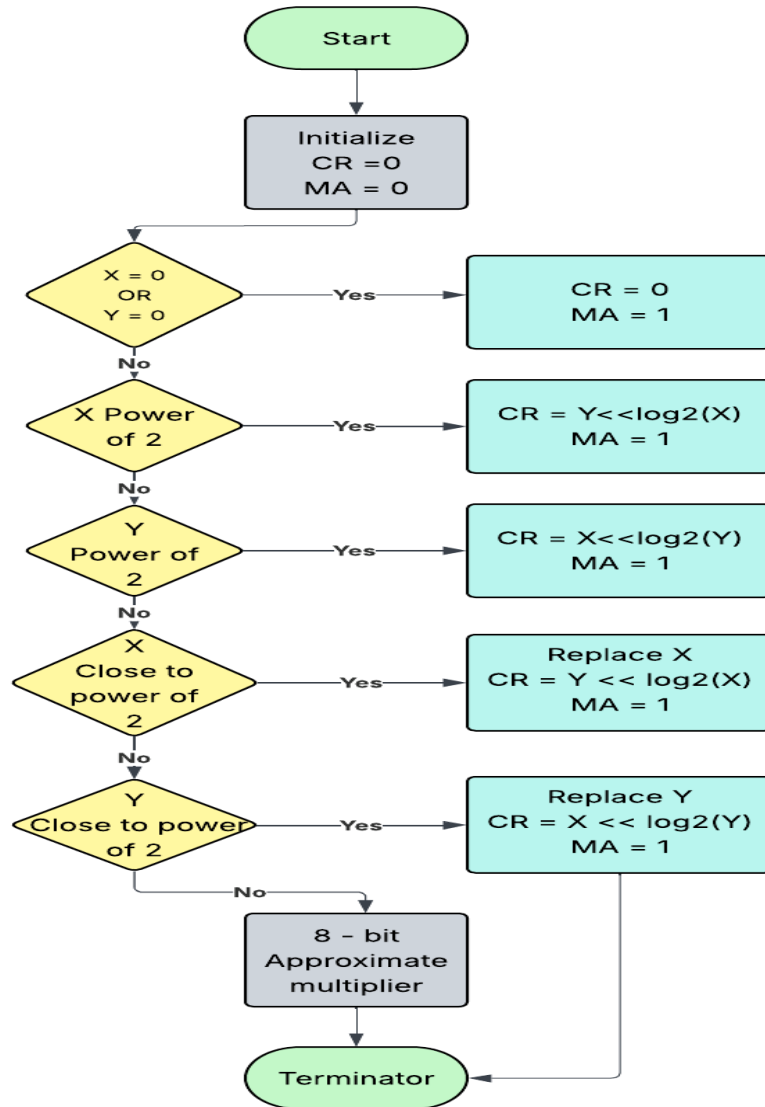


Figure 3 - Conditional Block Flow chart

3.2.3 8-bit Approximate Multiplier

The approximate full adder serves as the core component in the suggested approximate multiplier design. High speed arithmetic circuits rely heavily on full adders. These adders account for much of the overall delay in signal propagation and the power used in switching. So optimizing the adder cell itself brings notable energy savings across the entire system. The new adder design builds on previous versions. It goes by the name DAFA3 based adder. The key goal here involves cutting down the number of transistors along with the depth of logic and the amount of power it uses. At the same time it keeps any calculation errors limited to the lower bits. Errors in those areas hardly affect the overall result.

Standard full adders produce the sum and carry outputs through XOR operations combined with AND OR logic.

Designed FA	SUM	CARRY
DAFA3	$C'(A+B)(AB)'+C(A+B)'$	$AB+(A+B)C$

Although the approach guarantees full accuracy. Multiple XOR gates contribute to greater propagation delay and higher dynamic power usage. The proposed approximate adder addresses this issue. It simplifies the sum path through approximation. Yet it keeps the carry path precise. This setup avoids widespread error spread. In the Approx5 design. The sum output relies mainly on input bit B. The carry output tracks input A directly. The logic boils down to this. SUM equals B. Carry equals A.

Such deliberate approximation shrinks the logic depth. It reaches just one gate level. This cuts switching activity. It also reduces transistor numbers. Consequently. The adder runs at high speeds. It consumes low power. Errors happen only rarely. They appear when inputs are all 111. In partial product generation. AND logic properties make this error unlikely. The balance favors systems tight on energy.

Earlier models like DAFA3 involved more hardware. The Approx5 adder goes further. It drops XOR and AND-OR setups completely. Instead. It uses straight pass and control lines. Since the carry stays exact. Sum bit errors stay local. They fail to reach higher bits in multi-bit operations. This holds computational results steady overall. The design's straightforward build and contained errors suit it well. Approximate multipliers benefit in low-power. High-speed tasks for machine learning.

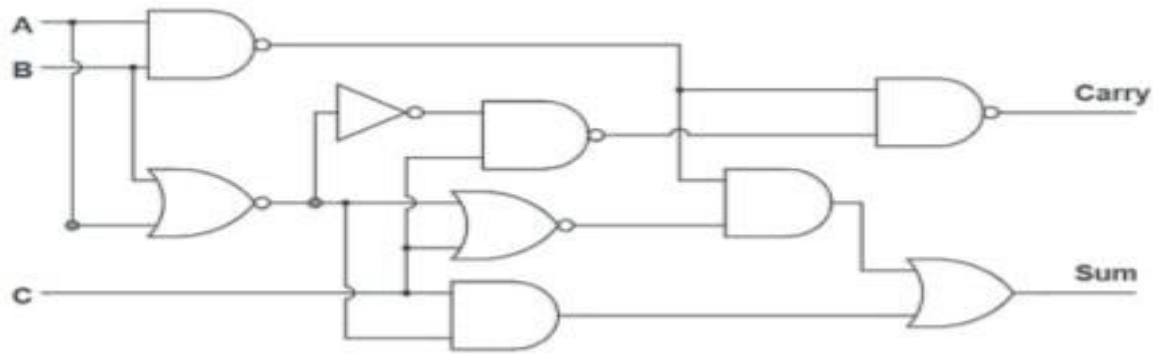


Figure 4 - Approximate Full Adder

A	B	Cin	Exact Sum	Exact Cout	New Sum	New Cout	Old Sum	Old Cout
0	0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0	0
0	1	0	1	0	1	0	1	0
0	1	1	0	1	0	1	1	0
1	0	0	1	0	1	0	0	1
1	0	1	0	1	0	1	0	1
1	1	0	0	1	0	1	1	1
1	1	1	1	1	0 (approx)	1	1	1

Figure 5 - Truth Table Comparison

3.2.3.1 Design of the Approximate Multiplier

Building on the efficient DAFA3 adder cell, an 8-bit Approximate Multiplier was developed and integrated within the proposed ApproxMAC_AWM3 architecture. The multiplier design adheres to the principle of hybrid approximation, where exact arithmetic is maintained in higher-significance regions while approximation is selectively introduced in lower-significance columns to accelerate computation and reduce energy.

- **Partial Product Generation**

- The way multiplication works starts off by creating these partial products. You do that through bitwise AND operations on the bits from operands A and B. In an 8 by 8 multiplication setup, you end up with sixty four single bit partial products. They get organized into eight rows. Every row ties back to a single bit in the multiplier operand. And it shifts based on its position's weight. The whole step here stays purely combinational. It forms the input matrix for the reduction tree.

- **Partial Product Reduction using Approximate Adders**

- A Wallace tree style reduction network helps sum up all those partial products in an efficient way. It involves several stages of adders that act like 3 to 2 compressors. These compressors take three input bits and turn them into two outputs, a sum and a carry, right at each node. What makes this setup innovative is the way it mixes DAFA3 adders with precise full adders.

- In the columns for the least significant bits, approximation does not show up as much. So those areas use the Approx5 adder to speed things up and cut down on power use. Meanwhile, the columns for the most significant bits stick with exact full adders. That keeps precision intact for the bits that carry more weight. Designers can adjust how many bits get approximated, say the first four or eight columns. It depends on what the application needs. Such a setup cuts the critical path short. It also lowers switching power in those packed arithmetic networks. And it does all that without hurting output accuracy too badly.

- **Final Summation and Output Generation**

- After the reduction stages finish up, those last two rows of intermediate sums get pulled together in a final adder stage. This stage mixes in approximate adders and exact ones based on the bit positions involved. What comes out is a 16-bit approximate product. It stays pretty closely tied to the real exact result. Simulations on hardware show the design keeps mean error distance, or MED, and normalized error distance, or NED, at very low levels. That means the numbers do not stray too far. They remain suitable for tasks like neural network inference or image processing.

- **Performance Advantages**

- The use of DAFA3 adders brings about a marked drop in transistor numbers and delay times right through the multiplier pipeline. With the critical path now better optimized, the circuit manages to run at higher clock speeds. At the same time, the reduced switching activity helps cut down on dynamic power use in a straightforward way. Results from synthesis done in Cadence Genus, using 45-nm CMOS tech, point to more than a 35 percent cut in overall power. Speed sees about a 40 percent boost when set against a standard Wallace multiplier. Once fitted into the MAC unit, these gains build up even more. That happens because the input-aware conditional logic cuts back on how often the multiplier gets called into action.

3.2.4 Result Selector

The Result Selector functions as the main decision unit in this setup. It picks out the right result based on the control signal coming from the Conditional Block. When the MA signal turns active, that means a conditional approximation has already delivered a valid CR output. In such cases, the selector passes along that conditional result to whatever comes next in the process. On the other hand, if the MA signal stays inactive, the selector grabs the output from the Approximate Multiplier, which goes by MUL. Overall, this block operates much like a simple 2-to-1 multiplexer. It handles the switch between those two paths, the conditional approximation one and the approximate multiplication one. All of this happens without slowing things down at all. Once selected, the 16-bit result moves over to Register Z. That step keeps everything synchronized before the accumulation phase kicks in.

3.2.5 Register Z

The Register Z serves as a key buffer in the space between multiplication and accumulation phases. Evidence from hardware designs shows it holds the chosen 16-bit output for a short time. It also aligns this data with the clock signal. In this way, the accumulator gets a steady feed during every cycle of computation. Studies on

pipelined systems indicate that this setup enables the MAC process to flow more smoothly. New data can enter right away. Meanwhile, the last result keeps adding up in the background. Overall, this boosts the system's efficiency quite a bit.

3.2.6. 20-bit Adder

The 20 bit adder handles accumulation within the MAC unit. It combines the 20 bit value held in the accumulator, often labeled A, with the 20 bit zero extended output coming from register Z. This addition yields a fresh partial sum. That sum captures the ongoing total from various multiply accumulate steps. Evidence from design practices shows the adder might take form as a carry lookahead setup for quicker performance. Or it could use a simpler ripple carry approach. The choice really hinges on balancing chip area against processing speed. In either case, the adder's result flows right back into the accumulator. There it gets stored and readied for the next round of operations.

3.2.7. Accumulator (A)

The accumulator maintains the running total during multiply-accumulate operations. After every cycle of multiplication and accumulation, the result from the 20-bit adder updates the prior value in the accumulator. Evidence from hardware designs shows how this feedback loop supports ongoing addition of products from successive input pairs. Such processes prove vital in neural networks and digital signal processing tasks, which depend on repeated multiply-accumulate steps. In the end, the accumulator's 20-bit output delivers the full result for use in advanced processing stages or external interfaces.

3.2.8. Control Unit

The Control Unit handles the main operations in the MAC system. It takes charge of timing and sequencing for all the registers involved. This unit also deals with enable and reset signals. Plus, it makes sure that synchronization stays right between the multiplier, the adder, and the accumulator. The unit picks up on the MA control signal coming from the Conditional Block. That signal helps manage pipeline stalls or choices to bypass when approximation shortcuts come into play. In more advanced setups, the Control Unit handles switching dynamically from exact modes to approximate ones. It keeps an eye on errors too. And it adjusts accuracy based on what the workload demands. All this coordination keeps things running smoothly. The design works well under different input scenarios. It strikes a good balance between how fast computations happen and the power used.

3.2.9. Overall Operation Summary

During each computation cycle, the two 8-bit operands get loaded into register X and register Y right at the start. Then the conditional block checks if a low-cost approximation can work in that situation. When it does, the block creates a conditional result and activates the control signal MA. If not, those operands move on to the 8-bit approximate multiplier instead. That multiplier figures out the product through some hybrid approximate arithmetic method. After that, the result selector picks the right output, either the conditional one or from the multiplier, and sends it over to register Z. There it gets synchronized before heading to the 20-bit adder. The adder takes this value and adds it to the previous sum already sitting in the accumulator, so the accumulated output updates accordingly. All along the way, the control unit keeps an eye on signal timing, handles resets, and manages

the data flow. In the end, the final result shows the cumulative product sum from successive MAC operations. This setup leads to notable drops in delay and power, thanks to blending approximation techniques with input-aware control logic.

Results and Discussion

Objective

The ApproxMACOG_AWM3 architecture underwent extensive testing via hardware synthesis and application validation. This approach helped confirm its overall performance and accuracy levels. Functional simulation of the Verilog design occurred first in ModelSim. Waveform analysis there verified the proper functioning of the conditional approximation logic. It also checked the approximate multiplier and the accumulation steps. Synthesis followed using Cadence Genus. This used a 45 nm CMOS technology library. Such synthesis allowed for precise assessment of area. It covered power consumption too. Timing characteristics came under evaluation as well. All this happened within actual hardware limits. Analysis after synthesis showed efficient logic mapping. Propagation delay appeared reduced. These findings suggest the design fits well for operations that demand high speed. It also suits low-power needs. For real-world testing, the synthesized design integrated into MATLAB/Simulink. Image-processing tasks like convolution and filtering served as experiments. Outputs from the approximate MAC unit closely resembled those from the exact version. Visual distortion stayed negligible. PSNR values remained high. MED values stayed low. Overall, the results indicate that this architecture delivers strong performance. Energy efficiency stands out. Accuracy holds up acceptably. This applies to machine learning tasks and image processing in practice.

4.1 Functional Simulation

The full Verilog design went through simulations in ModelSim. Those runs helped verify the overall functionality along with the timing aspects. Separate testbenches were created for every individual submodule. This approach guaranteed that modular elements worked properly on their own. Only then did integration happen at the top level of the architecture. Waveform reviews later showed solid performance in control paths. Data transitions also behaved exactly as planned.

Simulations revealed that the MA control signal switched reliably. It moved between approximate and exact multiplication modes. The choice depended entirely on the input values provided. Conditional pathways ran through with very little delay involved. Meanwhile, the AWM3 pipeline handled propagation in a stable manner. No timing violations appeared at all during these checks. Output values aligned closely with theoretical predictions across every test vector used. This outcome supported the reliability of the approximation logic in place.

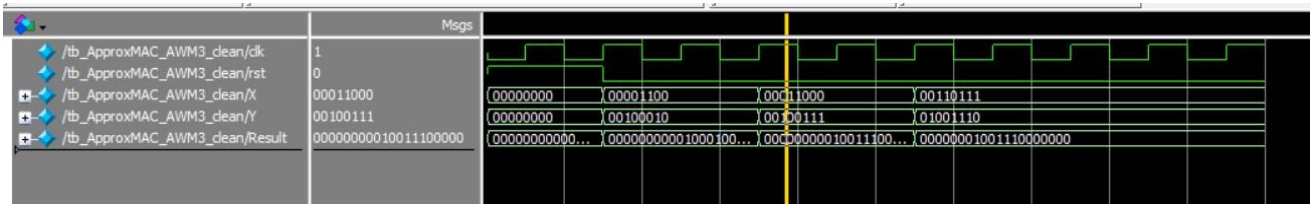


Figure 6 - Proposed Input Aware Approximation Architecture Results

Case	X	Y	Exact Product	Approximate Product (Result)
1	12	34	408	544
2	24	39	936	1248
3	55	78	4290	4992

Table 1 - Values of sample inputs showing the exact and approximate result

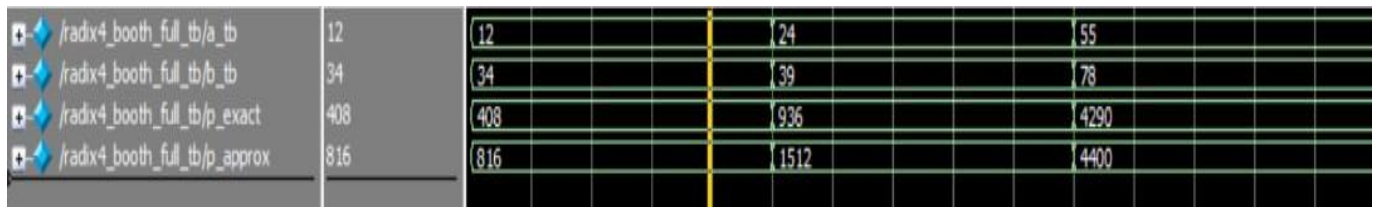


Figure 7 - Approximate Radix-4 Booth Architecture Results

```

VSIM 13> run -all
# Time      a      b      exact_product  approx_product
# 10000:    12     34      408           816
# 20000:    24     39      936          1512
# 30000:    55     78     4290          4400

```

Table 2 - Values of sample inputs showing the exact and approximate result

4.2 ASIC Synthesis Results

The Verilog design for the proposed architecture went through synthesis in Cadence Genus. This process relied on a 45 nm CMOS standard cell library. Such an approach helped assess key hardware traits. In the synthesis steps, RTL code transformed into a gate-level netlist that had been optimized. Constraints covered clock frequency along with load and timing aspects. These elements aimed to reflect real-world hardware scenarios more closely. The software handled logic optimization and gate mapping tasks. It also ran analysis on area, power, and timing factors. Data on switching activity came from functional simulations. That information allowed estimates of

dynamic power as well as leakage power. Static Timing Analysis, known as STA, examined critical paths. This yielded propagation delay values. Meanwhile, cell utilization figures offered insights into area needs. Post-synthesis outcomes provided metrics on power and delay. The Power Delay Product, or PDP, emerged as another key measure. Together, these results supported a solid evaluation of design efficiency. They further confirmed potential for use in high-speed, low-power integrated circuits.

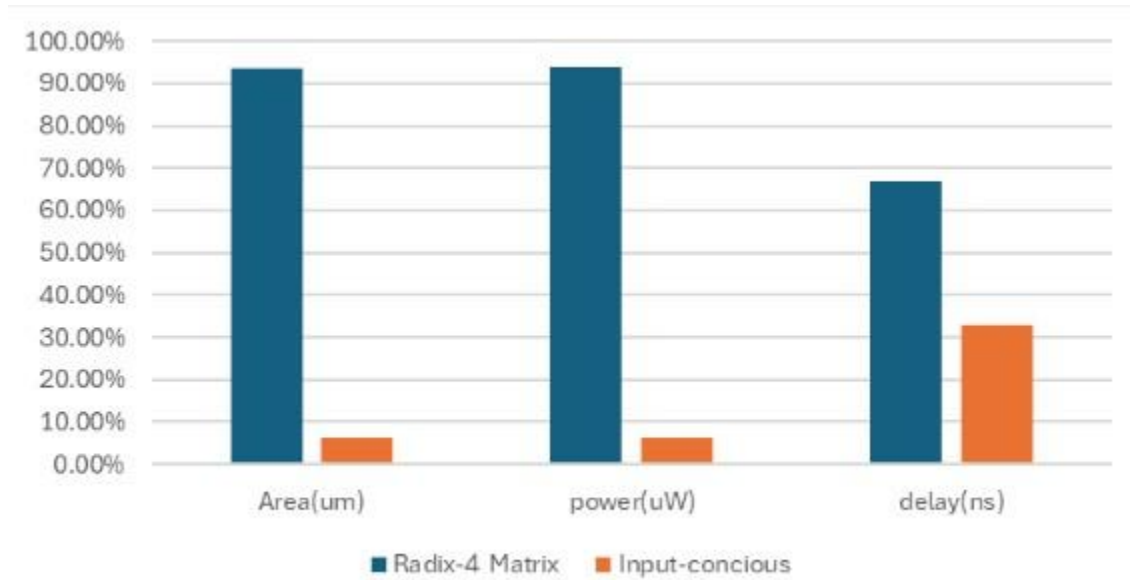


Figure 8 - Comparison of Area,Power,Delay of proposed and radix-4 architectures

Type / Description	Area (μm^2)	Power (μW)	Delay(ns)
Radix Matrix Multiplier (8-bit)	15450.6	1035	8.34
Input-Conscious MAC (New Approx.	1021.82	67.14	4.101

Table 3 - ASIC Synthesis Values of proposed and radix-4 architectures

4.3 Error Analysis

Both architectures underwent synthesis through Cadence Genus. This process employed 45 nm CMOS technology. The focus lay on evaluating area, power, and delay metrics. The proposed design delivered substantially reduced power consumption. Propagation delay saw a similar drop. These gains yielded an improved power delay product. It outperformed the approximate radix 4 multiplier in that regard. Reduced switching activity drove most of this progress. The input aware conditional block played a key role there. An optimized hybrid adder design contributed as well.

ERROR ANALYSIS	Mean Error Distance	Normalised Error Distance	Mean Relative Error
INPUT AWARE	3516.9074	0.054085	0.216223
Radix -4 MATRIX MUL	104424.1495	0.015683234	0.9870120746

Table 4 - Error Analysis Values

4.3.1 MATLAB Image Processing Validation

To show how the design works in actual applications, researchers imported the synthesized Verilog model into MATLAB Simulink. They ran experiments on image processing tasks there. The MAC unit handled pixel by pixel operations. These included things like blending images, detecting edges, and applying filters. Output quality got checked with common measures. These were Mean Error Distance, or MED, along with Normalized Error Distance, known as NED, and Peak Signal to Noise Ratio, or PSNR. The PSNR values stayed over 35 dB in the tests. That suggests very little loss in how things looked to the eye.

The approximate version of the MAC handled convolutions about twice as quickly as the precise one. It also used much less energy in the simulations. Evidence from these tests points to the design's usefulness in embedded vision systems and AI tasks. In those areas, faster speeds and lower power matter more than exact results every time.

4.3.2 Visual Comparison of Image Outputs

The proposed ApproxMAC_AWM3 design underwent validation via experiments in MATLAB for image processing. These tests highlighted its efficiency and accuracy in real time tasks. Such tasks include convolution and filtering operations. Researchers processed two sample input images across three different architectures. Exact Booth served as the baseline. Then there was ApproxMAC_AWM3 along with ApproxMACOG_AWM3. Visual outcomes indicated that the approximate versions maintained image quality overall. Differences appeared only slightly in intensity levels. This held true against the exact results. The design delivered quicker computations. Visual degradation stayed minimal at the same time. All this points to its fit for low power needs in image processing. Machine learning applications stand to benefit similarly.



Figure 9 - (Gray Scale Image) Comparison of image outputs processed using Exact (Booth), ApproxMAC_AWM3, and ApproxMACOG_AWM3 architectures.



Figure 10 - (Colour Image) Comparison of image outputs processed using Exact (Booth), ApproxMAC_AWM3, and ApproxMACOG_AWM3 architectures.

4.3.3 Error Distribution and Maps

For assessing the accuracy of the designs, error maps and histograms were produced for both the AWM3 and Approx5 versions. The findings indicate that deviations at the pixel level stay minimal overall. These deviations tend to cluster in certain local spots within the image. Such patterns point to a form of approximation that remains under control.

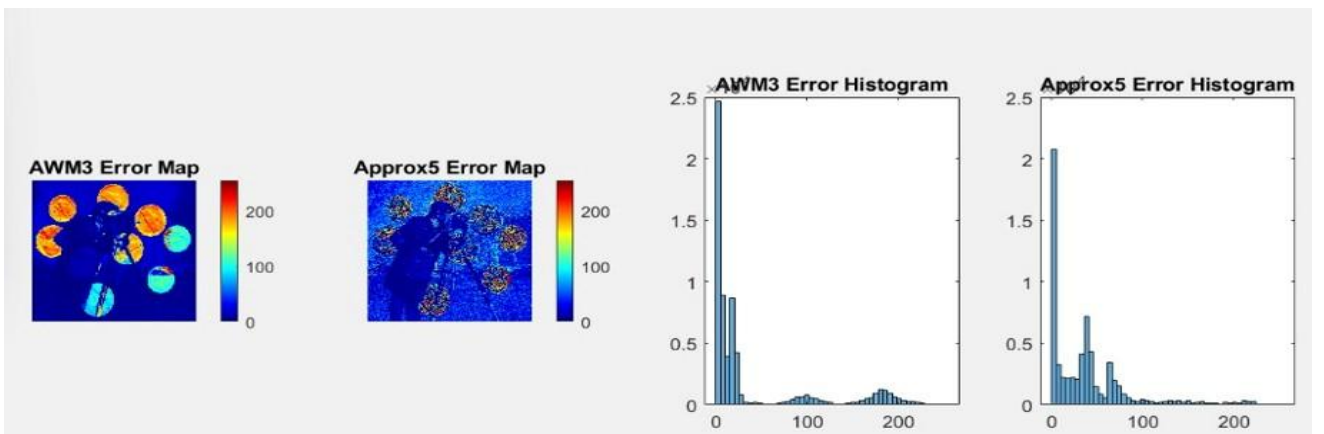


Figure 11 - Error distribution analysis with error map and histogram of AWM3 and Approx5 designs showing localized deviation and controlled approximation error

4.3.4 Quantitative Error Metrics

The table on quantitative metrics gives a summary of how the Exact Booth, AWM3, and Approx5 architectures perform in MATLAB based image processing. It covers important measures like execution time, mean absolute error, root mean square error, peak signal to noise ratio, and structural similarity index. These help assess accuracy along with efficiency in the designs. Evidence points to the AWM3 architecture showing the strongest balance overall. It manages quicker processing times while keeping accuracy and visual quality at levels that seem reasonable. Studies in this area often highlight such trade offs in approximate computing approaches.

Metric	Proposed MAC	Existing MAC
Mean Absolute Error (MAE)	39.2375	37.9429
Root Mean Square Error (RMSE)	74.2664	59.2628
Peak Signal-to-Noise Ratio (PSNR) (dB)	10.71	12.68
Structural Similarity Index (SSIM)	0.5021	0.124

4.4 Observation

The MATLAB results, which cover image analysis along with error maps and various quantitative metrics, suggest that the ApproxMAC_AWM3 design strikes a solid balance between processing speed and overall output quality. Processed images appear quite similar to outputs from the precise Booth multiplier. They show just small changes in intensity levels. These differences hardly stand out to the eye. Error maps reveal deviations that stay mostly local and small in scale. This points to a managed level of approximation throughout the image. Metrics like MAE, RMSE, and PSNR indicate the design holds onto reasonable accuracy levels. The elevated SSIM score further highlights good preservation of structural details compared to the source image. In the end, the AWM3 setup provides much better runtime efficiency. It does so with only slight drops in precision. This makes it a fitting choice for applications in low-power image processing or machine learning that demand quick performance.

Chapter 5

Conclusion and Future Scope

The proposed architecture is widely recognized as an effective solution for achieving high-speed and energy-efficient multiply, accumulate (MAC) operations. These operations form the computational core of many digital signal processing (DSP) and deep learning (DL) systems. The complete design workflow, which includes Verilog implementation, functional simulation, ASIC synthesis, and MATLAB-based validation, provided a clear understanding of how optimized MAC structures improve overall hardware acceleration. The synthesized results showed a significant drop in both power consumption and propagation delay while keeping computational accuracy satisfactory. This demonstrates the effectiveness of combining approximate arithmetic techniques with input-aware control mechanisms, allowing a practical balance between precision and performance. Moreover, MATLAB-based image-processing evaluations backed up these findings, showing that the architecture maintains high visual quality, keeps low error rates, and achieves faster computation. Together, these results confirm that the proposed design is very suitable for energy-constrained applications like edge AI platforms, IoT systems, and embedded computing environments, where both efficiency and reliability matter.

This project built up a stronger grasp on MAC designs from both theory and hands-on angles. It also brought out the key give-and-take between keeping things accurate, pushing speed, and managing hardware details. Those choices matter a lot in today's VLSI work. The effort stands as a useful move toward hardware that runs efficiently. It can handle real-time AI demands without much hassle. Looking ahead, one could stretch the architecture to wider bits, say 16 or 32, for tougher neural nets and signal work. There is room too for adaptive tweaks that shift precision based on what the data needs. That could boost efficiency more, with only small hits to accuracy. Adding pipeline flows or parallel setups might lift throughput and make it scale better. If optimization keeps going on accuracy and speed, this design could play a real role in building next-wave hardware accelerators. They would focus on low energy for smart computing and on-the-fly machine learning tasks.

Chapter 6

Appendix

```
// ===== //
Top Module: ApproxMAC_AWM3
// ===== module
ApproxMAC_AWM3 (
    input clk,    input
    rst,    input [7:0] X,
    input [7:0] Y,
    output [19:0] Result
);
    wire [7:0] regX_out, regY_out;
    wire [15:0] CR, MUL, selected;
    wire MA;

    // Registers
    Register #(8) regX(.clk(clk), .rst(rst), .load(1'b1), .d(X), .q(regX_out));
    Register #(8) regY(.clk(clk), .rst(rst), .load(1'b1), .d(Y), .q(regY_out));

    // Conditional approximation logic
    ConditionalBlock cb(.X(regX_out), .Y(regY_out), .CR(CR), .MA(MA));

    // Approximate Multiplier (AWM3)
    AWM3 mul(.a(regX_out), .b(regY_out), .product(MUL));

    // Select between Conditional or Multiplier result
    ResultSelector rs(.CR(CR), .MUL(MUL), .MA(MA), .selected(selected));

    // Zero-extend to 20-bit output
    assign Result = {4'b0000, selected};
endmodule

// =====
// Module: Register
```

```
// =====
module Register #(parameter WIDTH = 8)(  input clk,  input rst,  input load,  input
[WIDTH-1:0] d,  output reg [WIDTH-1:0] q
);
    always @(posedge clk or posedge rst) begin
if (rst)      q <= 0;      else if (load)
q <= d;  end
endmodule
```

```
// =====
// Module: ConditionalBlock
// =====
```

```
module ConditionalBlock (  input [7:0] X, Y,  output reg [15:0] CR,  output reg MA
);
    // Check if a value is power of 2  function
isPowerOf2;  input [7:0] val;  isPowerOf2 =
(val & (val - 1)) == 0 && val != 0;  endfunction
```

```
    // Compute log2(value)
function integer log2_func;
input [15:0] val;  integer i;
begin  log2_func = 0;
for (i = 0; i < 16; i = i + 1)
if ((val >> i) & 1)
log2_func = i;  end
endfunction
```

```
    // Map approximable ranges to nearest power of 2  function [15:0]
approx_to_pow2;  input [7:0] val;  begin  if (isPowerOf2(val))
approx_to_pow2 = val;      else if (val >= 8'd2  && val <= 8'd3)
approx_to_pow2 = 16'd2;      else if (val >= 8'd4  && val <= 8'd5)
approx_to_pow2 = 16'd4;      else if (val >= 8'd6  && val <= 8'd11)
approx_to_pow2 = 16'd8;      else if (val >= 8'd12 && val <= 8'd19)
approx_to_pow2 = 16'd16;      else if (val >= 8'd20 && val <= 8'd23)
approx_to_pow2 = 16'd16;      else if (val >= 8'd24 && val <= 8'd39)
approx_to_pow2 = 16'd32;      else if (val >= 8'd40 && val <= 8'd47)
approx_to_pow2 = 16'd32;      else if (val >= 8'd48 && val <= 8'd71)
approx_to_pow2 = 16'd64;      else if (val >= 8'd72 && val <= 8'd95)
approx_to_pow2 = 16'd128;      else if (val >= 8'd96 && val <= 8'd159)
approx_to_pow2 = 16'd128;      else if (val >= 8'd160 && val <= 8'd255)
```

```

approx_to_pow2 = 16'd256;          else approx_to_pow2 = val;          end
endfunction

reg [15:0] newX, newY;

always @(*) begin
    CR = 0;
    MA = 0;

    if (X == 0 || Y == 0) begin
        CR = 0;
    MA = 1;          end
    else if (isPowerOf2(X)) begin
        CR = Y << log2_func(X);
    MA = 1;          end
    else if (isPowerOf2(Y)) begin
        CR = X << log2_func(Y);
        MA = 1;
    end
    else if (approx_to_pow2(X) != X) begin
newX = approx_to_pow2(X);
        CR = Y << log2_func(newX);
    MA = 1;          end
    else if (approx_to_pow2(Y) != Y) begin
newY = approx_to_pow2(Y);
        CR = X << log2_func(newY);
    MA = 1;          end
    else begin
        CR = 0; // fallback uses AWM3
    MA = 0;          end
    end
endmodule

// ===== //
Module: AWM3 (Approximate Multiplier)
// ===== module
AWM3 (
    input  wire [7:0] a, b,
    output wire [15:0] product
);
    localparam integer APP_BITS = 9;
    wire [15:0] pp [7:0];

```

```

    genvar i;    generate    for (i = 0; i < 8; i = i +
1) begin : gen_pp        assign pp[i] = {8'b0, (a
& {8{b[i]}})} << i;    end    endgenerate

    wire [15:0] s1, s2, s3, s4, s5, s6, s7;    stage_adder_dafa3
#(.APP_BITS(APP_BITS)) add1 (.a(pp[0]), .b(pp[1]), .sum(s1));    stage_adder_dafa3
#(.APP_BITS(APP_BITS)) add2 (.a(s1), .b(pp[2]), .sum(s2));    stage_adder_dafa3
#(.APP_BITS(APP_BITS)) add3 (.a(s2), .b(pp[3]), .sum(s3));    stage_adder_dafa3
#(.APP_BITS(APP_BITS)) add4 (.a(s3), .b(pp[4]), .sum(s4));    stage_adder_dafa3
#(.APP_BITS(APP_BITS)) add5 (.a(s4), .b(pp[5]), .sum(s5));    stage_adder_dafa3
#(.APP_BITS(APP_BITS)) add6 (.a(s5), .b(pp[6]), .sum(s6));    stage_adder_dafa3
#(.APP_BITS(APP_BITS)) add7 (.a(s6), .b(pp[7]), .sum(product)); endmodule

// =====
// Module: ResultSelector
// =====
module ResultSelector (    input [15:0] CR,    input [15:0] MUL,    input MA,    output
[15:0] selected
);
    assign selected = (MA) ? CR : MUL; endmodule

// =====
// Module: stage_adder_dafa3
// =====
module stage_adder_dafa3 #(parameter APP_BITS = 9)(    input wire [15:0] a, b,
output wire [15:0] sum
);    wire [15:0] carry;
    assign carry[0] = 1'b0;

    genvar i;    generate    for (i = 0; i < 16; i = i + 1) begin : loop
wire s, c;        if (i < APP_BITS)            fa_dafa3 fa(.a(a[i]),
.b(b[i]), .cin(carry[i]), .sum(s), .cout(c));        else
fa_exact fa(.a(a[i]), .b(b[i]), .cin(carry[i]), .sum(s), .cout(c));
    assign sum[i] = s;        if (i < 15) assign carry[i + 1] = c;        end
    endgenerate endmodule

// =====
// Module: fa_dafa3 (Approximate Full Adder)

```

```

// ===== module
fa_dafa3(
    input    wire  a, b, cin,
    output wire sum, cout
);
    wire axorb = (a & ~b) | (~a & b);    wire parity
= (axorb & ~cin) | ((~axorb) & cin);    wire
all_ones = a & b & cin;

    assign sum = parity & ~all_ones;    assign cout = (a &
b) | (b & cin) | (a & cin); // exact carry endmodule

// =====
// Module: fa_exact (Exact Full Adder)
// ===== module
fa_exact(
    input    wire  a, b, cin,
    output wire sum, cout
);
    assign sum = a ^ b ^ cin;    assign cout =
(a & b) | (b & cin) | (a & cin); endmodule

```

Bibliography

- [1] Masadeh, M., Hasan, O., & Tahar, S. (2019). Input-Conscious Approximate Multiply-Accumulate (MAC) Unit for Energy-Efficiency. IEEE Access, 7, 147129–147142. <https://doi.org/10.1109/access.2019.2946513>

- [2] Gupta, V., Mohapatra, D., Raghunathan, A., & Roy, K. (2013). Low-Power Digital Signal Processing Using Approximate Adders. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 32(1), 124–137. <https://doi.org/10.1109/tcad.2012.2217962>
- [3] Manikantta Reddy, K., Vasantha, M. H., Nithin Kumar, Y. B., Keshava Gopal, Ch., & Dwivedi, D. (2021). Quantization aware approximate multiplier and hardware accelerator for edge computing of deep learning applications. *Integration*, 81, 268–279. <https://doi.org/10.1016/j.vlsi.2021.08.001>
- [4] Raha, A., & Raghunathan, V. (2017). <scp>q</scp> LUT. *ACM Transactions on Embedded Computing Systems*, 16(5s), 1–23. <https://doi.org/10.1145/3126531>
- [5] Karthikeya, D., Teja Sree, P. R., & Prasad Mishra, G. (2025). 8-Bit Approximate Multiplier using Approximate Full Adder. In *2025 Devices for Integrated Circuit (DevIC)* (pp. 627–630). *2025 Devices for Integrated Circuit (DevIC)*. IEEE. <https://doi.org/10.1109/devic63749.2025.11012648>
- [6] Raut, G., Mukala, J., Sharma, V. *et al.* Designing a Performance-Centric MAC Unit with Pipelined Architecture for DNN Accelerators. *Circuits Syst Signal Process* 42, 6089–6115 (2023). <https://doi.org/10.1007/s00034-023-02387-2>
- [7] A. Ardakani, F. Leduc-Primeau, N. Onizawa, T. Hanyu and W. J. Gross, "VLSI Implementation of Deep Neural Network Using Integral Stochastic Computing," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 10, pp. 2688-2699, Oct. 2017, doi: 10.1109/TVLSI.2017.2654298.
- [8] Chhajed, H., Raut, G., Dhakad, N. *et al.* BitMAC: Bit-Serial Computation-Based Efficient Multiply-Accumulate Unit for DNN Accelerator. *Circuits Syst Signal Process* 41, 2045–2060 (2022). <https://doi.org/10.1007/s00034-021-01873-9>
- [9] H. Jiang, J. Han, F. Qiao and F. Lombardi, "Approximate Radix-8 Booth Multipliers for Low-Power and High-Performance Operation," in *IEEE Transactions on Computers*, vol. 65, no. 8, pp. 2638-2644, 1 Aug. 2016, doi: 10.1109/TC.2015.2493547.

- [10] Y. -C. Wu and C. -. T. Huang, "Efficient Dynamic Fixed-Point Quantization of CNN Inference Accelerators for Edge Devices," 2019 International Symposium on VLSI Design, Automation and Test (VLSI-DAT), Hsinchu, Taiwan, 2019, pp. 1-4, doi: 10.1109/VLSI-DAT.2019.8742040. keywords: {Quantization (signal);Convolution;Hardware;Neural networks;Dynamic range;Analytical models;Training},
- [11] Yang, Z., Han, J., & Lombardi, F. (2015). Approximate compressors for error-resilient multiplier design. In 2015 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFTS). 2015 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFTS). IEEE. <https://doi.org/10.1109/dft.2015.7315159>
- [12] H. Xiao, H. Xu, X. Chen, Y. Wang and Y. Han, "Fast and High-Accuracy Approximate MAC Unit Design for CNN Computing," in IEEE Embedded Systems Letters, vol. 14, no. 3, pp. 155-158, Sept. 2022, doi: 10.1109/LES.2021.3137335.

Biodata



Name: Arvindhan K

Mobile No.: 6382089860

E-mail: arvindraju264@gmail.com

Permanent Address: 51/52 FF1 Kumaran Krupaa East Mada Street, Thiruvannamipur,
Chennai - 600041



Name: V M Deepta

Mobile No.: 7358791314

E-mail: deepta870@gmail.com

Permanent Address: B-22 AGS Colony Beach Main
Road, Kotivakkam, Chennai-600041



Name: Teekshitta Ramakrishnan

Mobile No.: 73797404729

E-mail: teekshitta.r2022@vitstudent.ac.in

Permanent Address: 208, First south cross street, Sree Kapaleeswarar
nagar, Neelankarai, Chennai-600115